

Global and local search

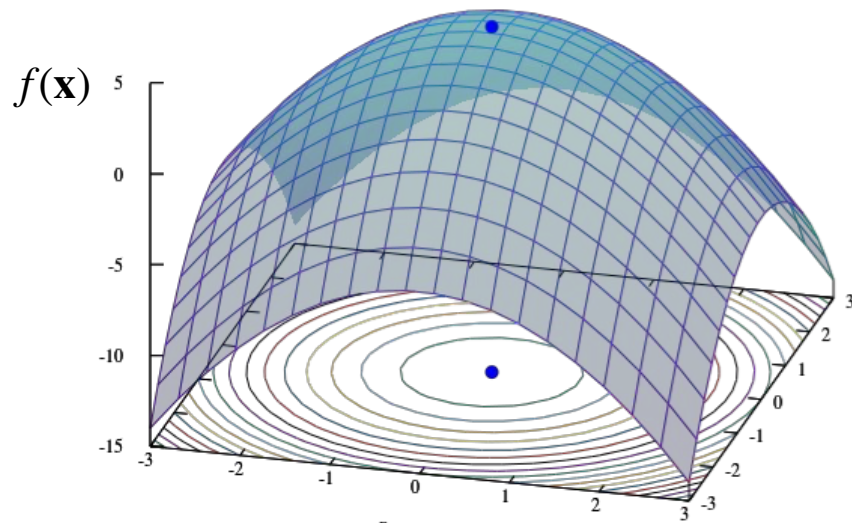
Stochastic optimization

$$\mathbf{x}^* = \operatorname{argmax}(f(\mathbf{x}))$$

- Gradients may not be available
- Evaluation of $f(x)$ may be expensive

For differentiable loss functions – *gradient descent*

For not differentiable unknown loss functions – *local and global search*



Global vs. Local Search

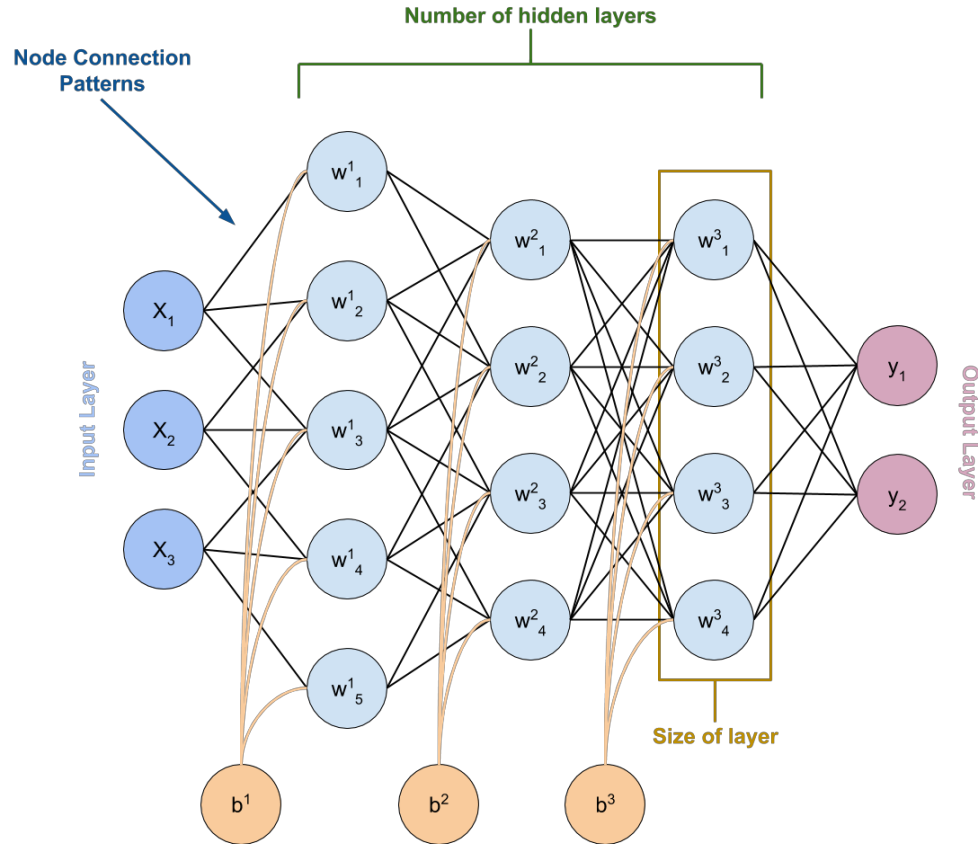
- **Global search** does not analyze a local neighborhood of a point.

Examples: grid search, random search, random walk, Bayesian optimization, cross-entropy optimization

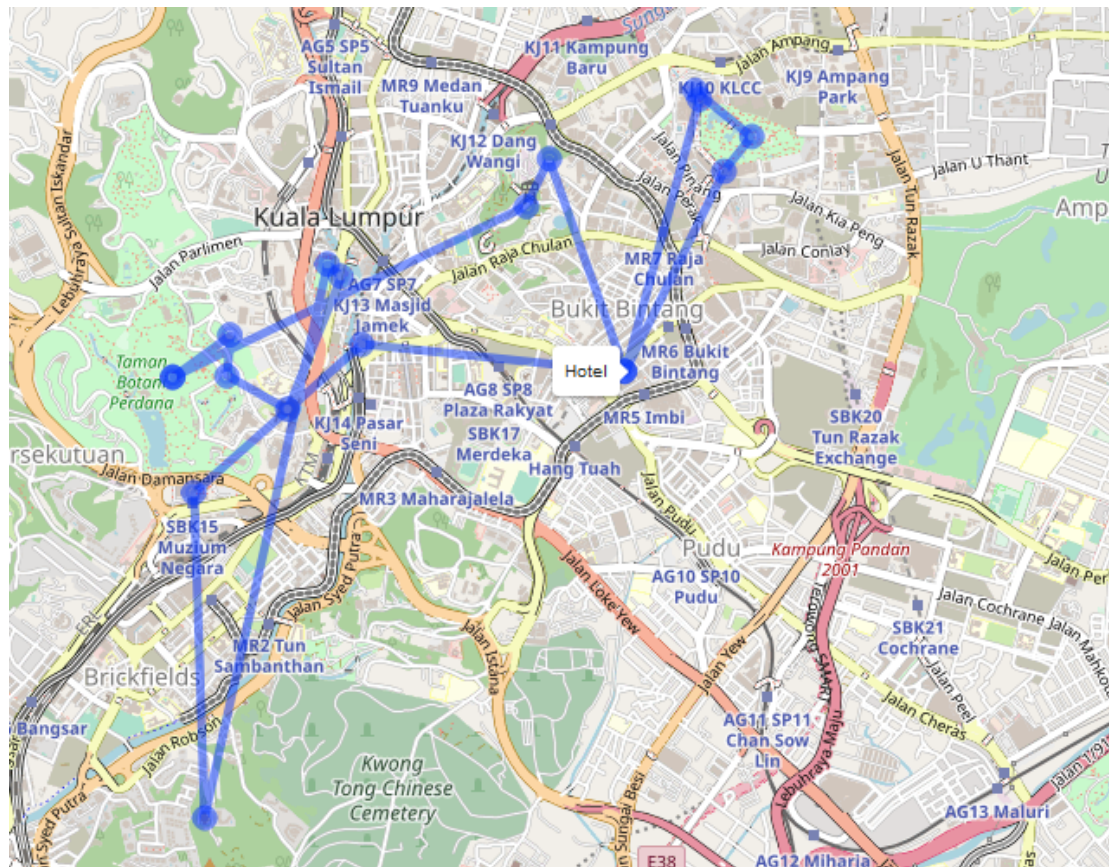
- **Local search** – choose a direction in the local neighborhood of a point.

Examples: hill climb, simulated annealing, genetic algorithm

Example: Hyperparameter Optimization

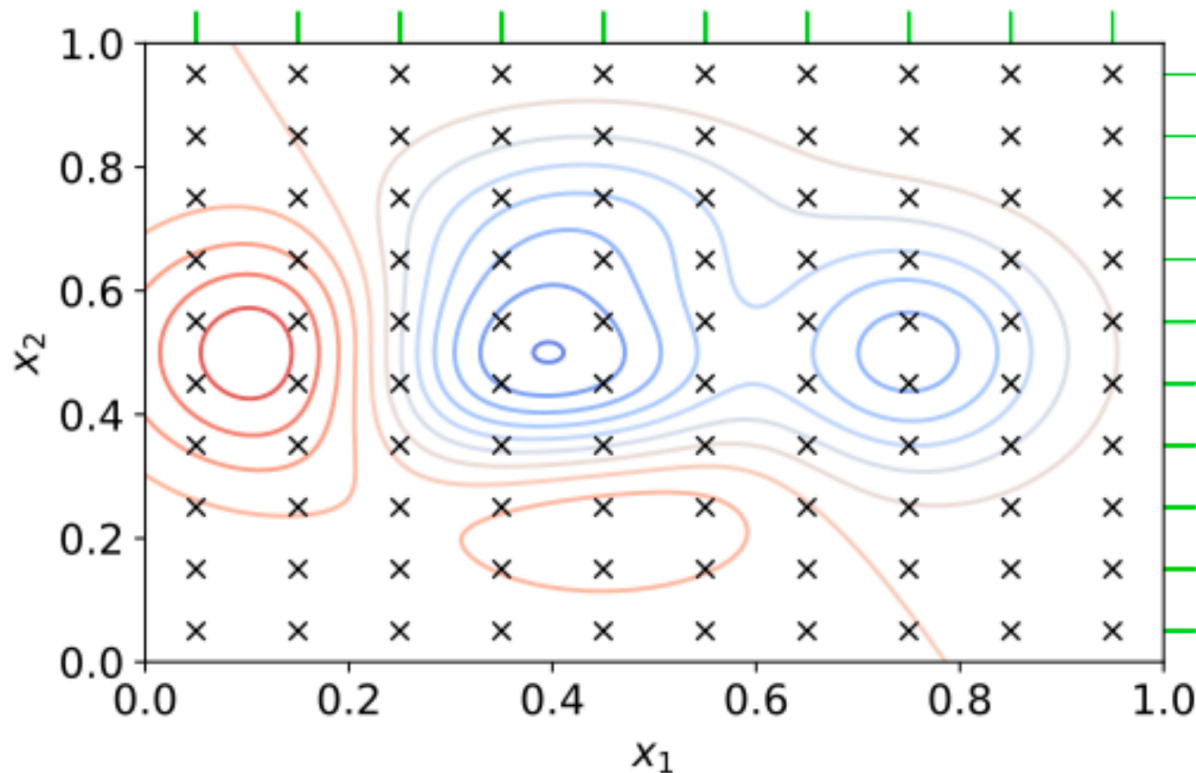


Example: Traveling Salesman Problem (TSP)



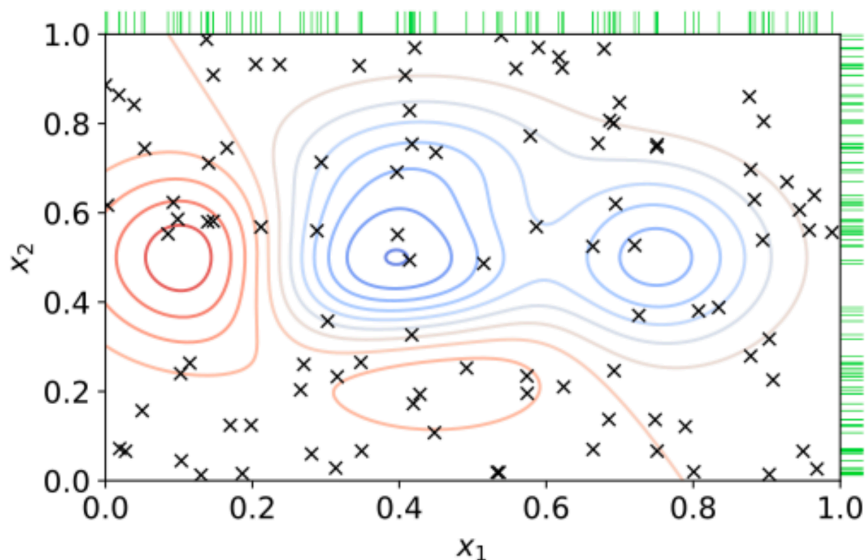
Grid Search

Sample with given intervals along all features and pick the best



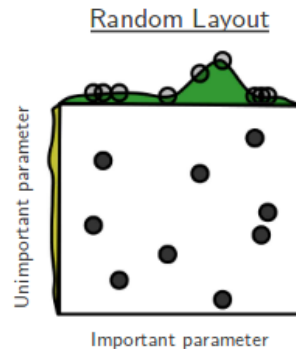
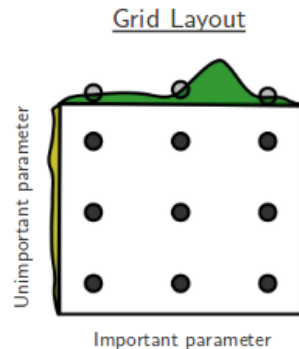
Random Search

Query the feature space many times at random and pick the best



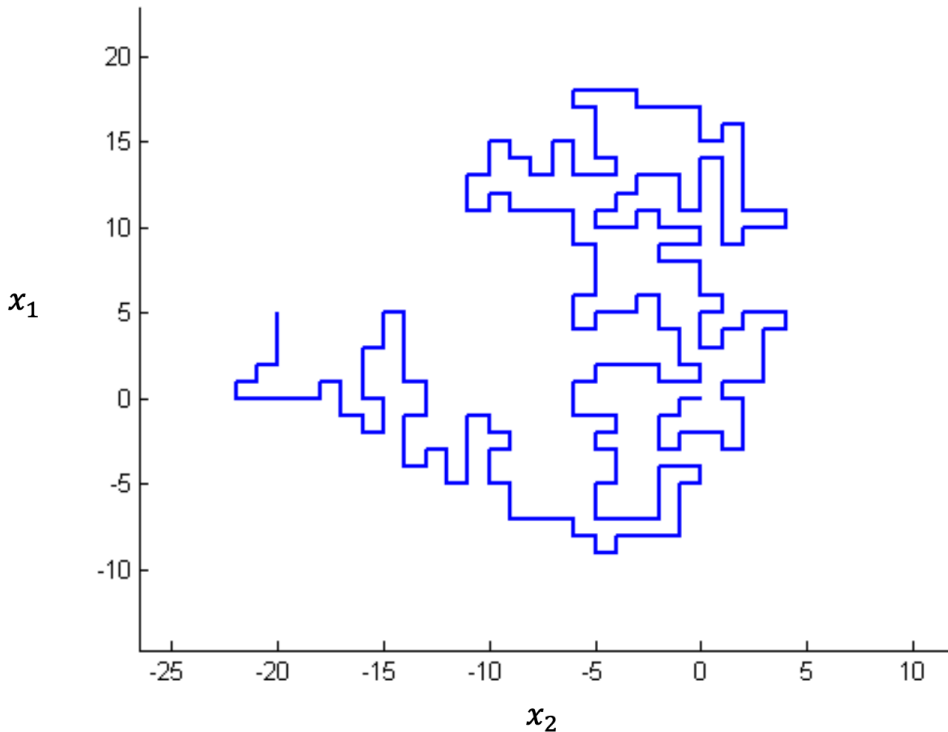
Advantages:

- Can miss out close-to optimal region with a regular grid, but have a high chance to locate it at random with appropriate amount of trials
- Faster when there is many parameters to tune and search space is large



Random Walk

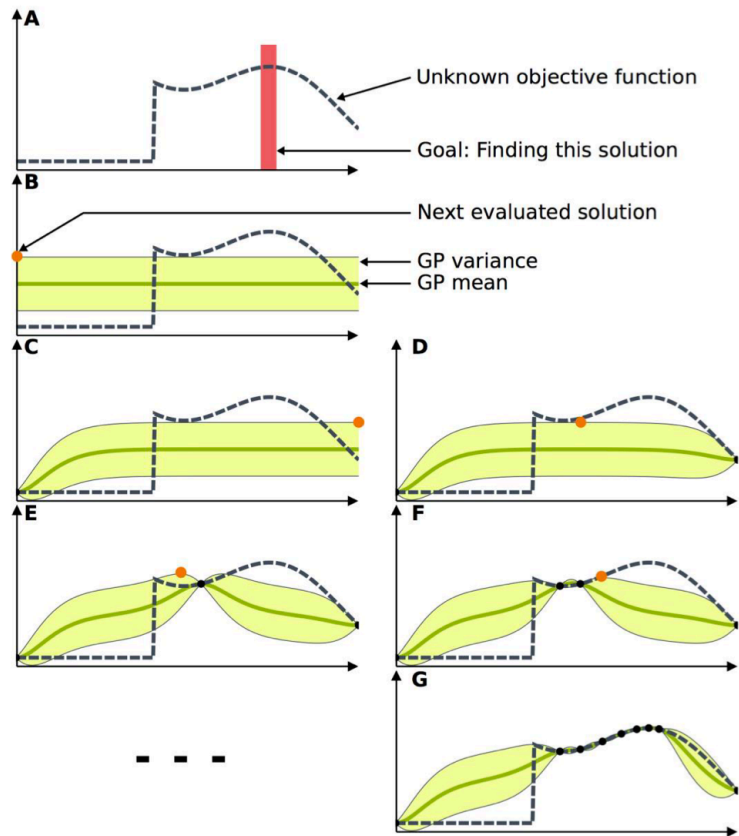
Same as random search, but we do not have the ability to sample independently from previous point



Bayesian optimization

General algorithm:

1. Evaluate an initial sample of data (or choose some *prior measure*)
2. Initialize a *surrogate model* to approximate the true objective function
3. Get the *acquisition function* $\alpha(\mathbf{x})$ to decide where to take next evaluation
4. Optimize it:
$$\mathbf{x}_{next} = \arg \max \alpha(\mathbf{x})$$
5. Sample new data, update surrogate model (get *posterior measure*)
6. Repeat 3-6 until the evaluation budget is over



Bayesian Optimization: Surrogate Model

Common solution - *Gaussian Process*:

Any finite collection of inputs $x_1, x_2, \dots, x_n$ has a joint Gaussian Distribution

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')),$$

$m(x)$ - mean function, $k(x, x')$ — covariance function

$$f(x_t) \sim \mathcal{N}(m(x_t), k(x_t, x_t))$$

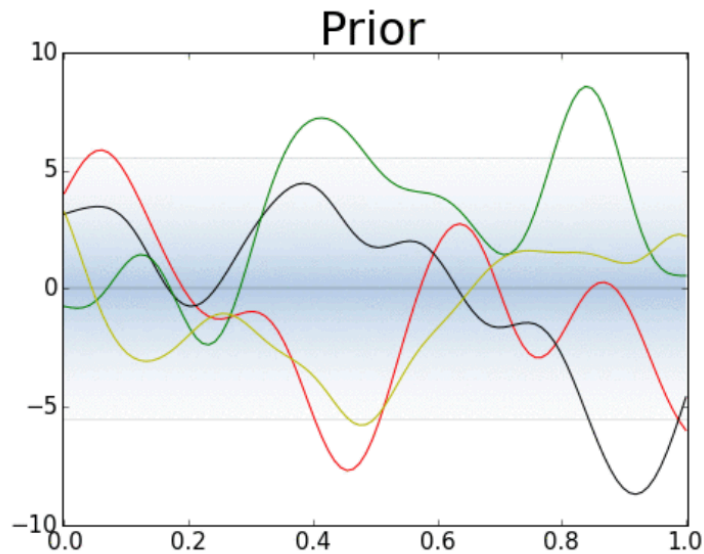
Prior Distribution:

$$m(x) = \mathbf{0}$$

The choice of k captures our belief about *smoothness* of the function

Common Choice:

$$k(x_i, x_j) = \exp\left(-\frac{1}{2} ||x_i - x_j||^2\right) \text{ — Square Exponent}$$



Bayesian Optimization: Surrogate Model

Posterior Distribution:

Given t samples $f_{1:t} = [f(x_1), f(x_2), \dots, f(x_t)]$ for a new point x_{t+1} we get:

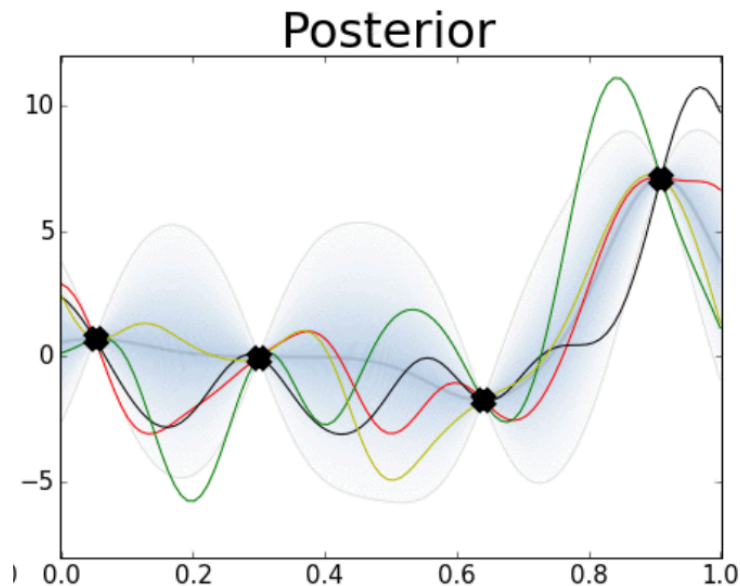
$$P(f_{t+1} | \mathcal{D}_{1:t}, x_{t+1}) = \mathcal{N}(\mu_t(x_{t+1}), \sigma_t^2(x_{t+1}))$$

$$\mu_t(x_{t+1}) = K_{t+1}^T (K + \sigma_{noise}^2 I)^{-1} f_{1:t}$$

$$\sigma_t(x_{t+1}) = K_{t+1,t+1} - K_{t+1}^T (K + \sigma_{noise}^2 I)^{-1} K_{t+1}$$

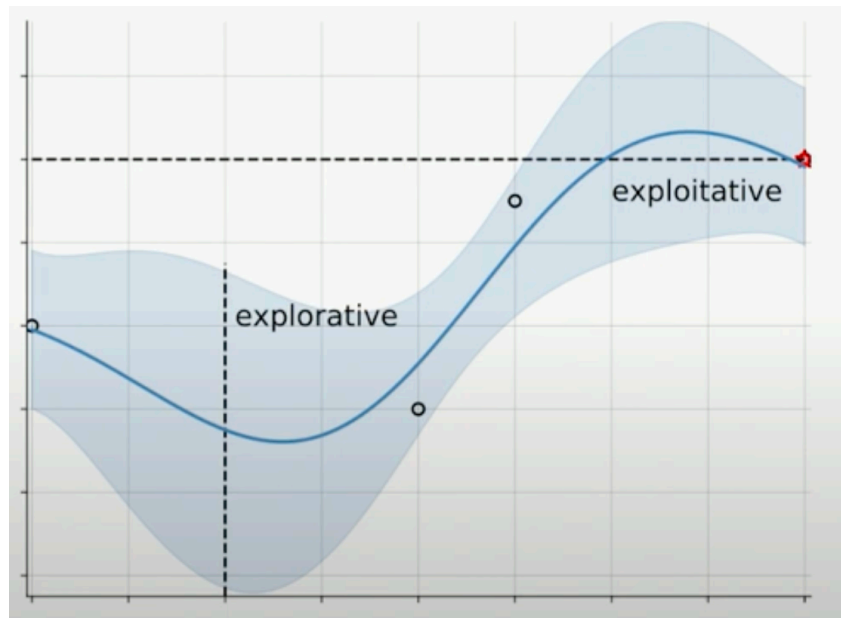
$$K = \begin{bmatrix} k(x_1, x_1) & \dots & k(x_1, x_t) \\ \vdots & \ddots & \vdots \\ k(x_t, x_1) & \dots & k(x_t, x_t) \end{bmatrix}$$

Posterior distribution is also Gaussian!



Bayesian Optimization: Acquisition Functions

Exploration vs. Exploitation



Bayesian Optimization: Acquisition Functions

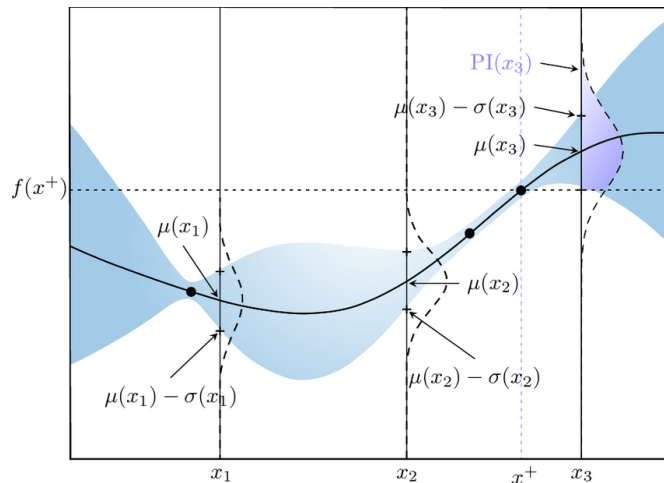
- **Probability of Improvement**

$$PI(x) = P[f(x) > y_{best} + \xi]$$

- Intuitive
- For a Gaussian can be evaluated as:

$$\alpha(x) = \Phi \left(\frac{\mu(x) - y_{best} - \xi}{\sigma(x)} \right)$$

ξ — exploration



- **Expected Improvement**

$$EI(x) = \mathbb{E} \max(f(x) - y_{best} - \xi, 0)$$

- *One of the most used*
- Less myopic

- **Upper Confidence Bound**

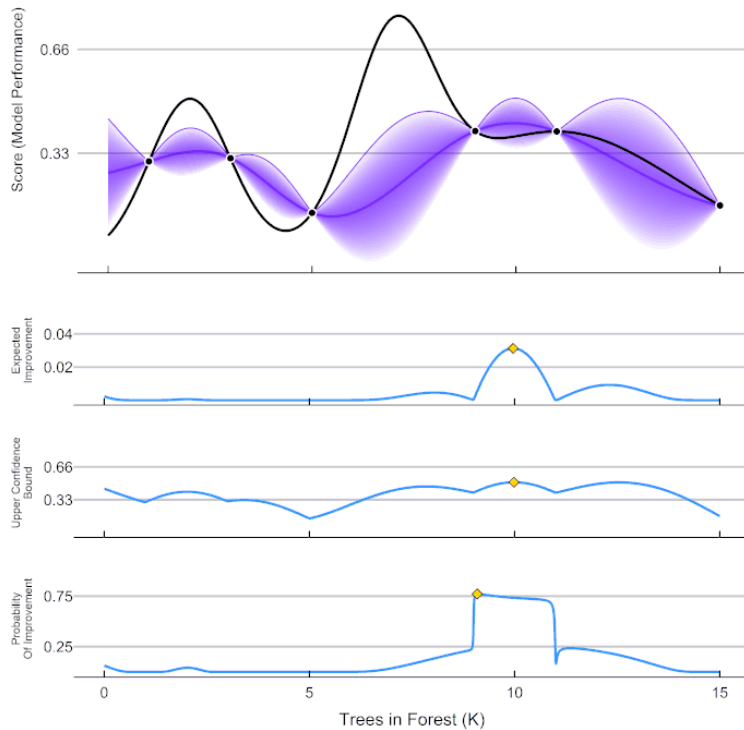
$$UCB(x) = q : P[f(x) < q] = 1 - \delta$$

- Straight tradeoff between exploration and exploitation
- For a Gaussian Case:

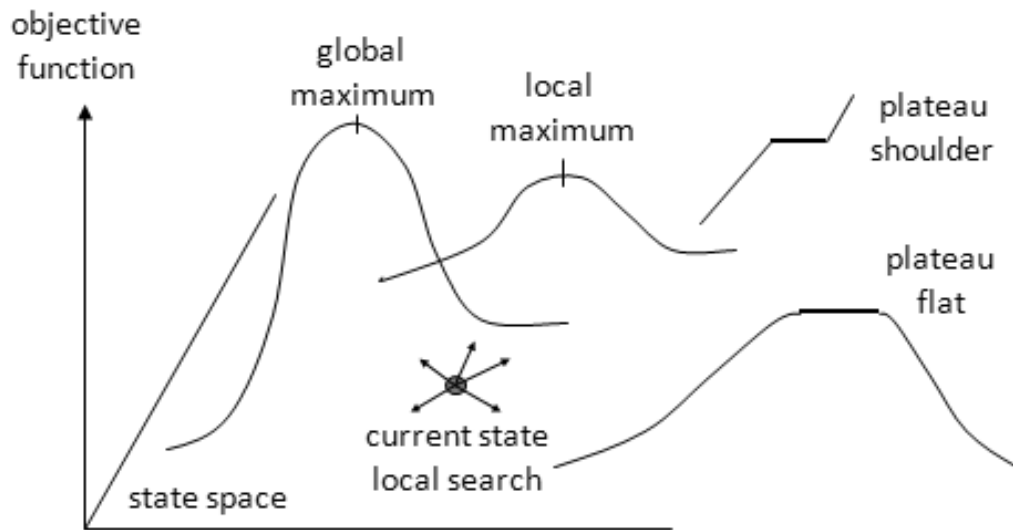
$$\alpha(x) = \mu(x) + \beta \sigma(x)$$

Bayesian Optimization: Acquisition Functions

ParBayesianOptimization in Action (Round 1)



Hill Climb



Similar to **Gradient Descent**

Calculate metric for possible shifts and step in a direction of best value

- **Stochastic Hill Climbing:**

Step with probability proportional to increase of metric (e.g. softmax)

- **Taboo Search**

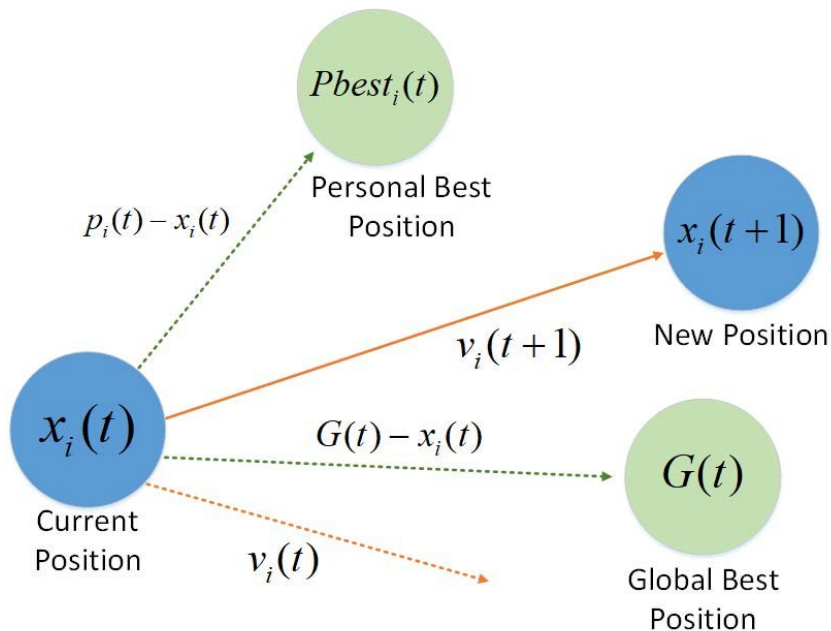
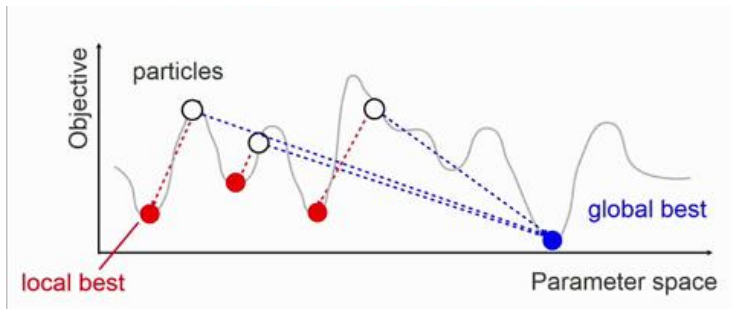
Do not return to the visited states

- **Particle Swarm Optimization**

Multiple «climbers» that pass information

Particle Swarm Optimization

New position is a combination of personal (local) best position and global best position

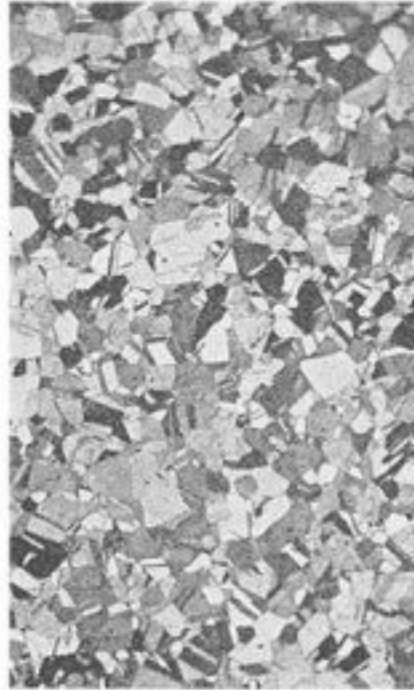


Simulated Annealing

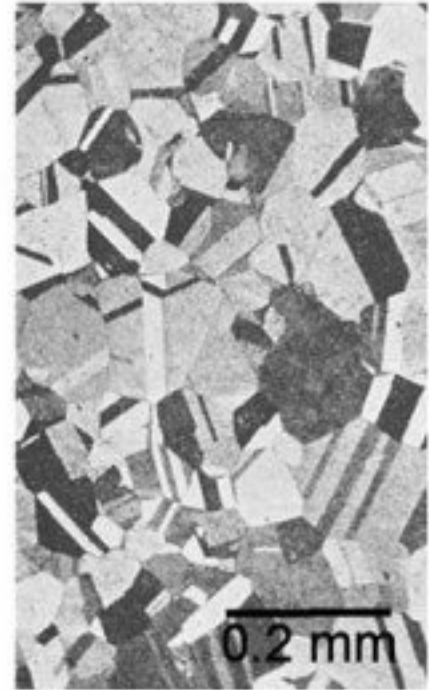
Rolling direction
↑



Cold-rolled



550 °C anneal for 1 hour



650 °C anneal for 1 hour

Annealing of metal

Simulated Annealing

Introduce temperature to stochastic hill climb

For Softmax:

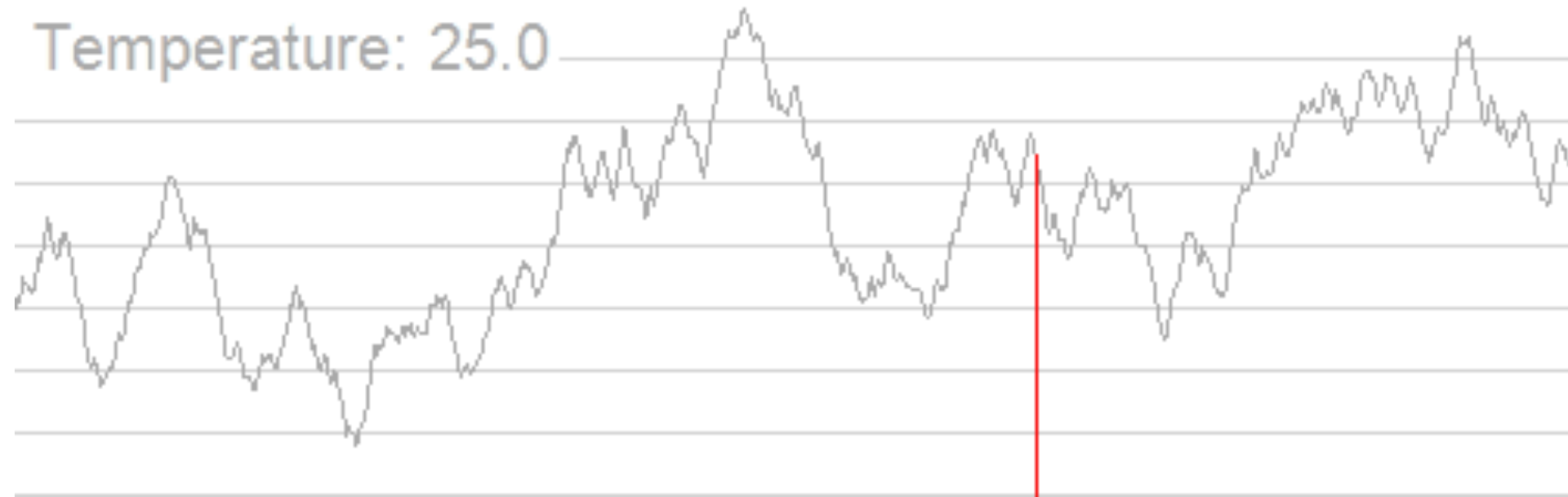
$$P(x_i) = \frac{e^{\frac{\Delta E(x_i)}{T}}}{\sum_j e^{\frac{\Delta E(x_j)}{T}}}$$

Start with high temperature (high exploration)
and gradually decrease (high exploitation)

	T - temperature			
	10000	10	1	0.1
ΔE	P(ΔE)	P(ΔE)	P(ΔE)	P(ΔE)
10	0.25016	0.43944	0.99325	1
5	0.25004	0.26653	0.00669	1.9287E-22
0	0.24991	0.16166	4.51E-05	3.7200E-44
-2	0.24986	0.13235	6.1E-06	7.6676E-53

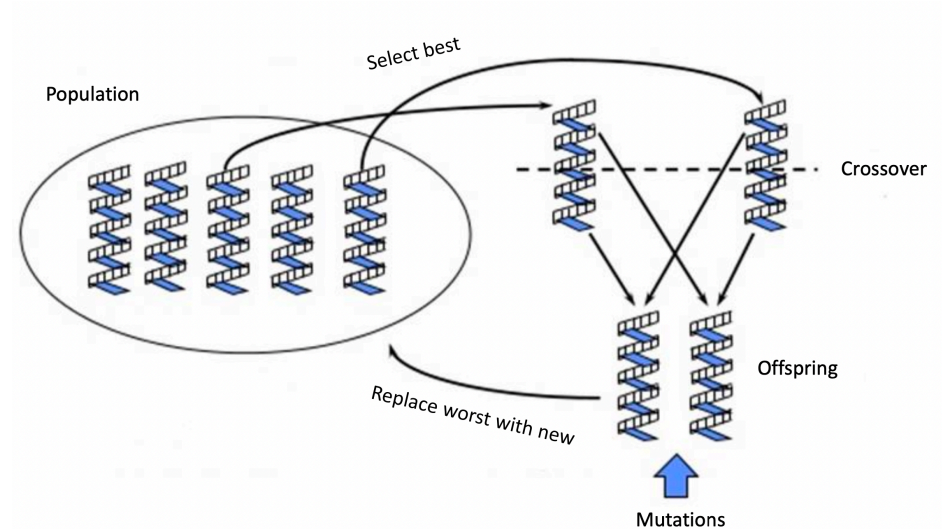
Simulated Annealing

Temperature: 25.0



Genetic Algorithm

- **Mutation** — variability operator that modifies a genotype
- **Crossover** — variability operator mixing genotypes
- **Fitness** — performance of a genotype with regard to objective
- **Generation** — points produced at iteration
- **Population** — all points existing at the moment

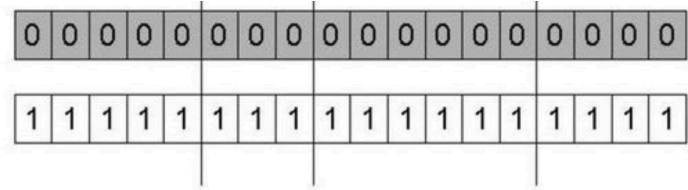


Genetic Algorithm: Crossover

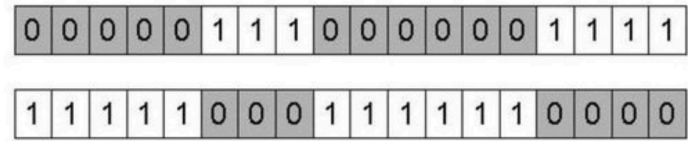
- **N-point Crossover**

- Choose n *random* crossover points
- Split along these points
- Glue parts, *alternating between parents*
- Inheritance is position dependent

parents



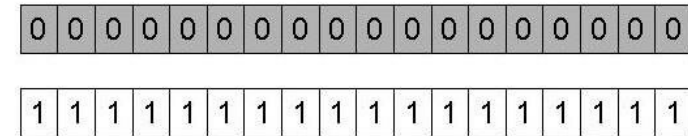
children



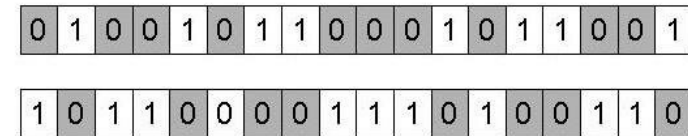
- **Uniform Crossover**

- Assign 'heads' to one parent, 'tails' to other
- *Flip a coin for each gene* of first child
- Make an *inverse copy* for second child
- Inheritance is position independent

parents



children



Genetic Algorithm: Crossover vs. Mutation

- Crossover is *explorative*: makes a *big* jump in the area between two parents
 - Mutation is *exploitative*: creates random small deviations, staying near the parent
-
- Only mutation can introduce *new information*

Genetic Algorithm: Evaluation and Selection

- Evaluate fitness of all the existing solutions
- Select individuals for survival (next iteration)
 - Rank-based (linear or exponential ranking)
 - Tournament-based (pick a group of individuals and select best of them)
- Can also add restriction on *percentage of population* to be replaced
 - Refresh whole population at each step
 - Replace one individual with one offspring at a step
 - Replace a proportion of population (e.g. 50%)
- *Elitism*: always keep at least one copy of the best solution so far